

Algoritmos Voraces



Fernando Tricas García – Javier Campos Laclaustra

Universidad de Zaragoza

Curso 2025/26

Presentación adaptada de la original de J. Campos

Este documento está sujeto a una licencia de uso Creative Commons. No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.





Índice

- 1 Introducción y primer ejemplo
- 2 El problema de la mochila
- 3 Consideraciones sobre la corrección del esquema voraz
- 4 Códigos de Huffman
- 5 Caminos mínimos en grafos
- 6 Árboles de recubrimiento de coste mínimo
- 7 El problema de la selección de actividades
- 8 El problema de la minimización del tiempo de espera
- 9 Fundamentos teóricos del esquema voraz
- 10 Un problema de planificación de tareas a plazo fijo
- 11 Heurísticas voraces



El esquema voraz: Introducción y primer ejemplo

- Es uno de los esquemas más simples y al mismo tiempo de los más utilizados.
- Típicamente se emplea para resolver problemas de optimización:
 - existe una **entrada** de tamaño n que son los candidatos a formar parte de la solución;
 - existen **subconjuntos** de esos n candidatos que satisfacen ciertas restricciones: se llaman **soluciones factibles**;
 - hay que obtener la solución factible que maximice o minimice una cierta función objetivo: se llama **solución óptima**.
- Ejemplos:
 - Encontrar la secuencia óptima para procesar un conjunto de tareas por un computador,
 - Hallar un camino mínimo en un grafo,
 - Problema de la mochila, ...



El esquema voraz: Introducción y primer ejemplo

Esquema genérico

```
1  algoritmo voraz(C: conjunto){
2      // C es el conjunto de todos los candidatos
3      S=∅;
4      //S es el conjunto en el que se construye la solución
5      mientras que not solución(S) and C!=∅ {
6          x = elemento de C que maximiza seleccionar(x);
7          C = C-{x};
8          si completable(S ∪ {x}) entonces S:=S ∪ {x}
9      }
10     si solución(S) entonces
11         devuelve S
12     sino
13         devuelve "no hay solución"
14 }
```



El esquema voraz: Introducción y primer ejemplo

Problema del cambio en monedas

- Se trata de devolver una cantidad de euros con el menor número posible de monedas.
- Se parte de:
 - un conjunto de tipos de monedas válidas, de las que se supone que hay cantidad suficiente para realizar el desglose, y de
 - un importe a devolver.

Imagen de: <https://www.profesionalhoreca.com/>

las-ventajas-de-las-cajas-registradoras-automaticas-para-controlar-el-efectivo-en-un-negocio-hostelero/





El esquema voraz: Introducción y primer ejemplo

Problema del cambio en monedas

- Elementos fundamentales del esquema:
 - **Conjunto de candidatos:** cada una de las monedas de los diferentes tipos que se pueden usar para realizar el desglose del importe dado.
 - **Solución factible:** un conjunto de monedas devuelto tras el desglose y cuyo valor total es igual al importe a desglosar.
 - **Completable:** la suma de los valores de las monedas escogidas en un momento dado no supera el importe a desglosar.
 - **Función de selección:** elegir, si es posible, la moneda de mayor valor de entre las candidatas.
 - **Función objetivo:** número total de monedas utilizadas en la solución (debe minimizarse).



El esquema voraz: Introducción y primer ejemplo

```
1  tipo moneda=(M25,M10,M5,M1) // por ejemplo
2  ...
3  funcion cambia(importe:nat; valor:vector[moneda] de nat)
4      devuelve vector[moneda] de nat
5  variables mon:moneda; cambio:vector[moneda] de nat
6  {
7      para mon en moneda {
8          cambio[mon]:=0
9      }
10     para mon:=M25 hasta M1 {
11         mientras que valor[mon]<=importe {
12             cambio[mon]:=cambio[mon]+1;
13             importe:=importe-valor[mon]
14         }
15     }
16     devuelve cambio
17 }
```



El esquema voraz: Introducción y primer ejemplo

- Problemas sobre el problema del cambio en monedas:
 - Demostrar la corrección del algoritmo.
 - Demostrar, buscando contraejemplos, que el algoritmo no es óptimo si se añade un nuevo tipo de moneda de 12 euros o si se elimina alguno de los tipos existentes. Demostrar que, en esas condiciones, el algoritmo puede incluso no encontrar solución alguna aunque ésta exista.
- Otro problema:
 - ¿Es el método de ordenación por selección directa un algoritmo voraz?
 - Si lo es, ¿cuáles son las funciones utilizadas (“selección”, “completable”, “solución”)?



El problema de la mochila

- Se tienen n objetos **fraccionables** y una mochila.
- El objeto i tiene **peso** p_i , $1 \leq i \leq n$.
- La mochila tiene **capacidad** C .
- Si se introduce una **fracción** x_i , $0 \leq x_i \leq 1$, del objeto i en la mochila, entonces se consigue un **beneficio** $b_i x_i$.
- El objetivo es **llenar** la mochila de manera que se **maximice el beneficio** total.
- Pero como la mochila tiene capacidad C , el peso total de todos los objetos metidos en ella no puede **superar** esa cantidad.



El problema de la mochila

Formalmente:

$$\text{maximizar} \quad \sum_{1 \leq i \leq n} b_i x_i \quad (1)$$

$$\text{sujeto a} \quad \sum_{1 \leq i \leq n} p_i x_i \leq C \quad (2)$$

$$\text{con} \quad 0 \leq x_i \leq 1, b_i > 0, p_i > 0, 1 \leq i \leq n \quad (3)$$

- Una solución **factible** es cualquier n-tupla $(x_1, \dots, x_n)$ que satisfaga (2) y (3).
- Una solución **óptima** es cualquier solución factible para la que (1) sea máximo.
- Si $p_1 + \dots + p_n \leq C$, entonces, obviamente $x_i = 1, 1 \leq i \leq n$ es una solución óptima.
 - Por tanto, supongamos que $p_1 + \dots + p_n > C$.
- Nótese además que todas las soluciones óptimas llenarán la mochila por completo (podemos poner '=' en (2)).



El problema de la mochila

Ejemplo:

$$n = 3, C = 17$$

$$(b_1, b_2, b_3) = (40, 36, 22)$$

$$(p_1, p_2, p_3) = (15, 12, 8)$$

Tres soluciones factibles (entre otras muchas):

$$\frac{(x_1, x_2, x_3) \quad \sum_{1 \leq i \leq 3} b_i x_i}{(i) \quad (1, 1/6, 0) \quad 46}$$



El problema de la mochila

Ejemplo:

$$n = 3, C = 17$$

$$(b_1, b_2, b_3) = (40, 36, 22)$$

$$(p_1, p_2, p_3) = (15, 12, 8)$$

Tres soluciones factibles (entre otras muchas):

	(x_1, x_2, x_3)	$\sum_{1 \leq i \leq 3} b_i x_i$
(i)	(1, 1/6, 0)	46
(ii)	(0, 3/4, 1)	49



El problema de la mochila

Ejemplo:

$$n = 3, C = 17$$

$$(b_1, b_2, b_3) = (40, 36, 22)$$

$$(p_1, p_2, p_3) = (15, 12, 8)$$

Tres soluciones factibles (entre otras muchas):

	(x_1, x_2, x_3)	$\sum_{1 \leq i \leq 3} b_i x_i$
(i)	(1, 1/6, 0)	46
(ii)	(0, 3/4, 1)	49
(iii)	(0, 1, 5/8)	49.75



El problema de la mochila

Solución voraz:

- El conjunto de candidatos son los objetos, tomándose de ellos cierta fracción.
- Un conjunto de candidatos es:
 - **completable** si la suma de sus pesos no supera la capacidad de la mochila,
 - una **solución** si iguala dicha capacidad.

- La función objetivo a maximizar es:

$$\sum_{1 \leq i \leq n} b_i x_i$$

- La función de selección es la más difícil de determinar.
 - Si procedemos vorazmente, en cada paso debemos considerar un objeto y tomar cierta fracción suya.
 - La cuestión de qué fracción se toma es más fácil de resolver: si hemos elegido el mejor candidato, tomamos todo lo que podamos de él.



El problema de la mochila

- ¿Cuál es el mejor candidato (es decir, la mejor función de selección)?
- Volvamos al ejemplo:
 - Primera estrategia: elegir el objeto con **mayor beneficio total** (el primero). Sin embargo, la mochila se llena muy rápidamente con poco beneficio total.
 - Segunda estrategia: elegir el objeto que **llene menos la mochila**, para acumular beneficios de un número mayor de objetos. Sin embargo, es posible que se elija un objeto con poco beneficio simplemente porque pesa poco.
 - La tercera estrategia, que es la óptima, es tomar siempre el objeto que proporcione **mayor beneficio por unidad de peso**.
- Los algoritmos resultantes de aplicar cualquiera de las dos primeras estrategias también son voraces, pero no calculan la solución óptima.



El problema de la mochila

```
1  constante n=... // número de objetos
2  tipo vectReal=vector[1..n] de real
3  ...
4  //Pre:  $\forall i \in 1..n : peso[i] > 0 \wedge$ 
5  //       $\forall i \in 1..n-1 : benef[i]/peso[i] \geq benef[i+1]/peso[i+1]$ 
6  funcion mochila(benef,peso:vectReal; cap:real) devuelve vectReal
7  variables resto:real; i:entero; sol:vectReal
8  {
9      para i en 1..n {
10         sol[i]:=0.0 // inicializar solución
11     }
12     resto:=cap; // capacidad restante
13     i:=1;
14     mientras que (i<=n) and (peso[i]<=resto) {
15         sol[i]:=1; resto:=resto-peso[i]; i:=i+1
16     }
17     si i<=n { sol[i]:=resto/peso[i] }
18     devuelve sol
19 }
20 // Post: sol es solución óptima del problema de la mochila
```




El problema de la mochila

- Coste temporal: $\Theta(n)$
 - $\Theta(n \log(n))$ si se tiene en cuenta que hay que ordenar primero los vectores.

- Demostración de la corrección:

Sea $X = (x_1, \dots, x_n)$ la solución generada por el algoritmo voraz.

Si todos los x_i son iguales a 1, la solución es claramente óptima.

Luego, sea j el menor índice tal que $x_j \neq 1$.

Por tanto $x_i = 1$ para $1 \leq i < j$, $x_i = 0$ para $j < i \leq n$, y $0 \leq x_j < 1$.

Si X no es óptima ent. existe otra solución factible $Y = (y_1, \dots, y_n)$ tal que

$$\sum b_i y_i > \sum b_i x_i.$$

Sin pérdida de generalidad, asumimos que $\sum p_i y_i = C$.

Sea k el menor índice tal que $y_k \neq x_k$ (claramente, existe ese k).

Se verifica: $y_k < x_k$. En efecto:

- Si $k < j$ entonces $x_k = 1$. Pero $y_k \neq x_k$ luego $y_k < x_k$.
- Si $k = j$ entonces, puesto que $\sum p_i y_i = C$ y $y_i = x_i$ para $1 \leq i < j$, se sigue que $y_k < x_k$ ó $\sum p_i y_i > C$.
- Si $k > j$ entonces $\sum p_i y_i > C$, y eso no es posible.



El problema de la mochila

Ahora, supongamos que aumentamos y_k hasta x_k y disminuimos tantos $(y_{k+1}, \dots, y_n)$ como sea necesario para que la capacidad utilizada siga siendo C .

Se obtiene así una nueva solución $Z = (z_1, \dots, z_n)$

con $z_i = x_i$, para $1 \leq i \leq k$ y $\sum_{k < i \leq n} p_i(y_i - z_i) = p_k(z_k - y_k)$.

Entonces:

$$\begin{aligned} \sum_{1 \leq i \leq n} b_i z_i &= \sum_{1 \leq i \leq n} b_i y_i + (z_k - y_k) p_k b_k / p_k - \sum_{k < i \leq n} (y_i - z_i) p_i b_i / p_i \\ &\geq \sum_{1 \leq i \leq n} b_i y_i + [(z_k - y_k) p_k - \sum_{k < i \leq n} (y_i - z_i) p_i] b_k / p_k \\ &= \sum_{1 \leq i \leq n} b_i y_i \end{aligned}$$

Si $\sum_{1 \leq i \leq n} b_i z_i > \sum_{1 \leq i \leq n} b_i y_i$, entonces Y no puede ser óptima.

Si esas sumas son iguales entonces, bien $Z=X$ y X es óptima, o bien $Z \neq X$.

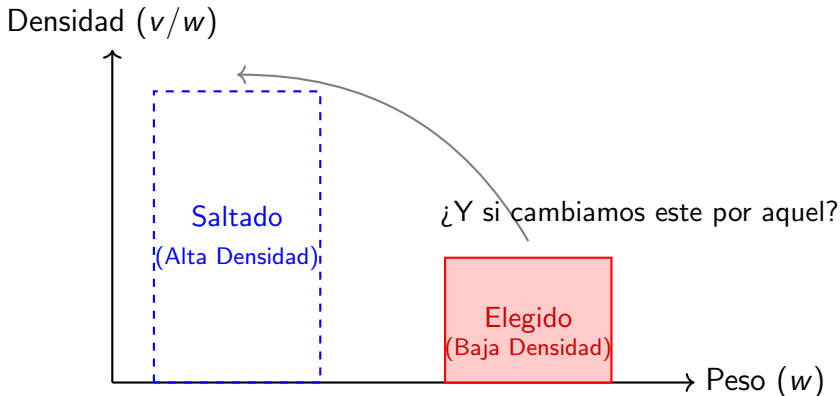
En este último caso, se usa de nuevo el mismo argumento hasta demostrar que, bien Y no es óptima, o bien Y se transforma en X , demostrando que X también es óptima. $\square$



Prueba de optimalidad: Argumento del Intercambio

Paso 1: Supongamos una solución mejor

que NO sigue la estrategia voraz (se salta el más valioso)



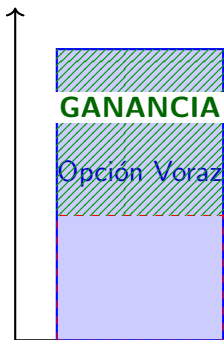


Prueba de optimalidad: Argumento del Intercambio

Paso 2: Realizamos el intercambio

Al coger el de mayor densidad, el área total aumenta.

Densidad (v/w)



El ancho (peso) es el mismo (w).

Pero como $h_{azul} > h_{rojo}$, el área azul es mayor.

Conclusión: La solución voraz siempre da más valor total.



Consideraciones sobre la corrección del esquema voraz

- La selección de una candidata óptima en cada paso es una **estrategia heurística** que no siempre conduce a la solución óptima.
- En ocasiones, se **utilizan heurísticas voraces** para obtener **soluciones subóptimas** cuando el cálculo de las óptimas es demasiado costoso.
- ¿Puede saberse que familias de problemas (de optimización u otros) pueden resolverse con una estrategia voraz?
La respuesta es: **NO** exactamente (aunque hay resultados).
- Sin embargo, existen ciertos indicios... más bien, técnicas de demostración:
 - la propiedad de la **selección voraz** y
 - la existencia de una **subestructura óptima**.



La propiedad de la selección voraz

- Se verifica esta propiedad cuando una solución **globalmente óptima** puede ser alcanzada mediante **selecciones localmente óptimas** que son tomadas **en cada paso** sin depender de las **selecciones futuras**;



La propiedad de la selección voraz

- Se verifica esta propiedad cuando una solución **globalmente óptima** puede ser alcanzada mediante **selecciones localmente óptimas** que son tomadas **en cada paso** sin depender de las **selecciones futuras**;
- en otras palabras, una estrategia voraz progresa de **arriba hacia abajo**, tomando una decisión voraz tras otra, **reduciendo** iterativamente el problema a otro más pequeño.



¿Cómo se comprueba si se verifica la propiedad de la selección voraz?

- Normalmente, se examina una solución **globalmente óptima**,
- Se trata de demostrar que esa solución puede ser **manipulada** de forma que se obtiene tras una **primera selección voraz (localmente óptima)**,
- Esa selección **reduce el problema** a otro similar pero más pequeño;
- Se aplica **inducción** para demostrar que se puede usar una **selección voraz** en cada paso
hay que demostrar que una solución óptima posee una “subestructura óptima”

Subestructura óptima:

- Un problema posee una **subestructura óptima** si una solución óptima de ese problema incluye **soluciones óptimas de subproblemas**.



Códigos de Huffman

D.A. Huffman

“A method for the construction of minimum-redundancy codes”,
Proceedings of the IRE, 40(9), pp. 1098-1101, 1952.

- Los códigos de Huffman son una técnica muy útil para comprimir ficheros.
- El algoritmo voraz de Huffman utiliza una tabla de frecuencias de aparición de cada carácter para construir una forma óptima de representar los caracteres con códigos binarios.



Códigos de Huffman. Ejemplo

- Se desea comprimir un fichero que tiene 100.000 caracteres. Las frecuencias de aparición de caracteres en el fichero son las siguientes:

	a	b	c	d	e	f
Apariciones (en miles)	45	13	12	16	9	5



Códigos de Huffman. Ejemplo

- Se desea comprimir un fichero que tiene 100.000 caracteres. Las frecuencias de aparición de caracteres en el fichero son las siguientes:

	a	b	c	d	e	f
Apariciones (en miles)	45	13	12	16	9	5

- Puede usarse un código de longitud fija (de 3 bits). Requeriría 300.000 bits.

	a	b	c	d	e	f
Codificación con longitud fija	000	001	010	011	100	101



Códigos de Huffman. Ejemplo

- Se desea comprimir un fichero que tiene 100.000 caracteres. Las frecuencias de aparición de caracteres en el fichero son las siguientes:

	a	b	c	d	e	f
Apariciones (en miles)	45	13	12	16	9	5

- Puede usarse un código de longitud fija (de 3 bits). Requeriría 300.000 bits.

	a	b	c	d	e	f
Codificación con longitud fija	000	001	010	011	100	101

- Puede hacerse mejor con un **código de longitud variable**, dando codificaciones cortas a los caracteres más frecuentes y más largas a los menos frecuentes.

	a	b	c	d	e	f
Codificación con longitud variable	0	101	100	111	1101	1100

Este código ahorra algo más del 25 % (requiere 224.000 bits).



Códigos de Huffman. Código **libre de prefijos**

Para que sea posible:

- Ninguna **codificación** puede ser **prefijo** de otra.
 - la **decodificación** es inmediata pues **no hay ambigüedades**.

Ejemplo:

001011101
sólo puede ser
aabe

	a	b	c	d	e	f
Codificación con longitud variable	0	101	100	111	1101	1100



Códigos de Huffman. Código **libre de prefijos**

Para que sea posible:

- Ninguna **codificación** puede ser **prefijo** de otra.
 - → la **decodificación** es inmediata pues **no hay ambigüedades**.

Ejemplo:

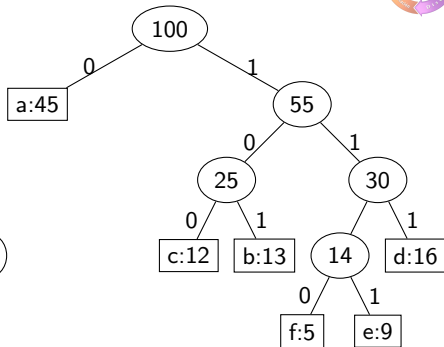
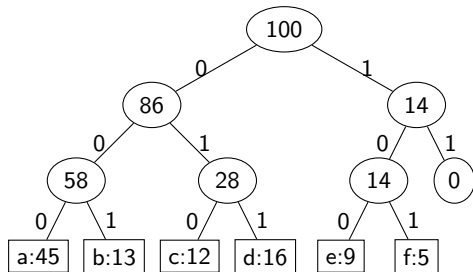
001011101
sólo puede ser
aabe

El código se representa mediante un **trie** (árbol lexicográfico):

- Árbol binario cuyas **hojas** son los caracteres codificados;
- El código de cada carácter es el **camino** desde la raíz hasta la hoja, donde:
 - Ir al hijo izquierdo significa '0', e
 - Ir hacia el derecho significa '1'.



Códigos de Huffman. Ejemplos.



- Cada **hoja** está etiquetada con un **carácter** y su **frecuencia**.
- Cada **nodo interno** está etiquetado con la **suma de los pesos** de las hojas de su subárbol.



Códigos de Huffman

Un **código óptimo** siempre está representado por un **árbol lleno**:
cada nodo interno tiene dos hijos.

Si el alfabeto que se quiere codificar es C , entonces el árbol del código óptimo tiene:

- $|C|$ hojas y
- $|C|-1$ nodos internos.

Códigos de Huffman. Construcción del árbol.



- El algoritmo voraz de Huffman construye el árbol A de un código óptimo de abajo hacia arriba.
- Utiliza una **cola** Q de árboles con **prioridades** (las frecuencias hacen de prioridades).
- Empieza con un conjunto de $|C|$ hojas en Q y realiza una secuencia de $|C|-1$ “mezclas” hasta crear el árbol final.
- En cada paso, se “mezclan” los dos objetos (árboles) de Q que tienen menos frecuencia y el resultado es un nuevo objeto (árbol) cuya frecuencia es la suma de las frecuencias de los dos objetos mezclados.

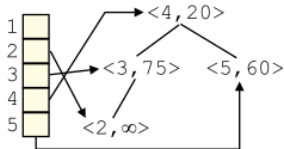


Cola con prioridades

genero cpa {colaPriAristas}
operaciones

creaVacía: $\rightarrow$ cpa	$\Theta(1)$
inserta: cpa vért etiq $\rightarrow$ cpa	$\Theta(\log n)$
primero: cpa $\rightarrow$ (vért,etiq)	$\Theta(1)$
borra: cpa $\rightarrow$ cpa	$\Theta(\log n)$
sustit: cpa vért etiq $\rightarrow$ cpa	$\Theta(\log n)$
valor: cpa vért $\rightarrow$ etiq	$\Theta(1)$
está?: cpa vért $\rightarrow$ bool	$\Theta(1)$
vacía?: cpa $\rightarrow$ bool	$\Theta(1)$

Implementación: montículo junto con un vector indexado por vértices



el valor ∞ de la etiqueta significa
que el vértice no está en la cola



Códigos de Huffman

```

1  // Pre: C es un conjunto de caracteres y f es el
2  //      vector de frecuencias de esos caracteres
3  funcion Huffman(C:conjunto;f:vectFrec) devuelve árbol
4  variables Q:colaPri; i,fx,fy,fz:entero; z,x,y:árbol
5  {
6      creaVacía(Q);
7      para todo x en C {
8          inserta(Q,<x,f[x]>)
9      }
10     para i:=1 hasta |C|-1 {
11         <x,fx>:=primero(Q); borra(Q);
12         <y,fy>:=primero(Q); borra(Q);
13         fz:=fx+fy;
14         z:=creaÁrbol(raíz=>fz, hijoIzq=>x, hijoDer=>y);
15         inserta(Q,<z,fz>)
16     }
17     <z,fz>:=primero(Q); borra(Q)
18     devuelve z
19 }
20 //Post: z es el árbol de un código libre de prefijos óptimo para
21 //      (C,f)

```

Códigos de Huffman. Para el ejemplo anterior

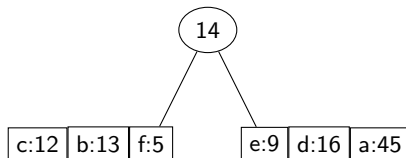


f:5	e:9	c:12	b:13	d:16	a:45
-----	-----	------	------	------	------



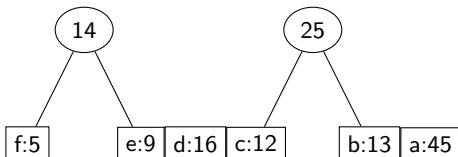
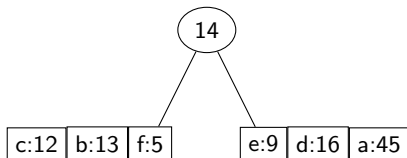
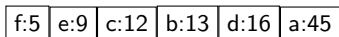
Códigos de Huffman. Para el ejemplo anterior

f:5	e:9	c:12	b:13	d:16	a:45
-----	-----	------	------	------	------



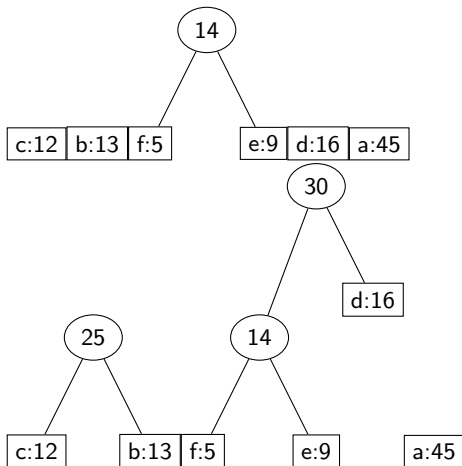
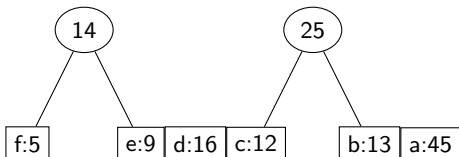
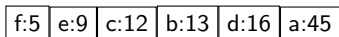


Códigos de Huffman. Para el ejemplo anterior

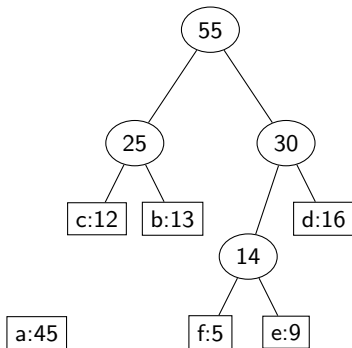




Códigos de Huffman. Para el ejemplo anterior

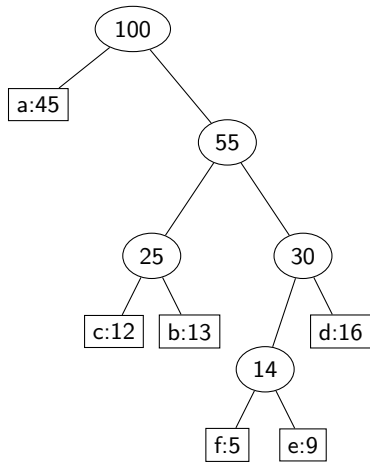
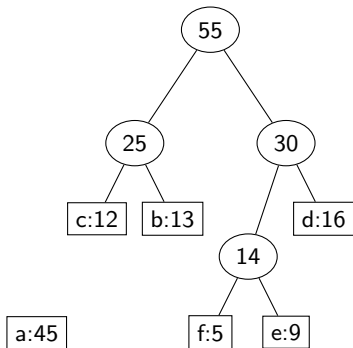


Códigos de Huffman. Para el ejemplo anterior





Códigos de Huffman. Para el ejemplo anterior





Códigos de Huffman. Coste temporal

- Inicialización de Q : $\Theta(|C|)$ (ejercicio)
- El interior del bucle: $\Theta(\log |C|)$

Coste total: $\Theta(|C| \log |C|)$



Códigos de Huffman. Coste temporal

- Inicialización de Q : $\Theta(|C|)$ (ejercicio)
- El interior del bucle: $\Theta(\log |C|)$

Coste total: $\Theta(|C| \log |C|)$

Corrección: ver que se verifican las propiedades de:

- Selección voraz y
- Subestructura óptima.

Códigos de Huffman. Propiedad de la selección voraz.



Sea C un alfabeto en el que cada carácter c tiene frecuencia $f[c]$. Sean x e y dos caracteres de C con frecuencias mínimas. Entonces existe un código libre de prefijos óptimo para C en el que las codificaciones de x e y tienen igual longitud y se diferencian sólo en el último bit.

La idea de la demostración consiste en tomar un árbol que represente un código libre de prefijos óptimo arbitrario y modificarlo hasta convertirlo en otro árbol que represente otro código libre de prefijos óptimo y tal que los caracteres x e y aparezcan como hojas hermanas de máxima profundidad. Detalles en [CLRS09, pp. 433-434].

Por tanto, el proceso de construir un árbol óptimo mediante mezclas puede empezar, sin pérdida de generalidad, con la elección voraz de mezclar los dos caracteres de menor frecuencia.



Códigos de Huffman. Propiedad de la subestructura óptima.

Sea T un árbol binario lleno que representa un código libre de prefijos sobre un alfabeto C , con frecuencia $f[c]$ para cada carácter $c \in C$.

Sean x e y dos caracteres de mínima frecuencia que aparecen como hojas hermanas en T , y sea z su padre.

Considerando z como un carácter con frecuencia $f[z] = f[x] + f[y]$, el árbol $T' = T \setminus \{x, y\}$ representa un código libre de prefijos para el alfabeto $C' = C \setminus \{x, y\} \cup \{z\}$.

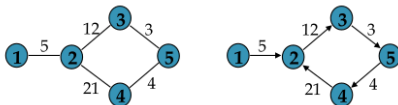
Si T' es óptimo para C' , entonces T es óptimo para C . La demostración, en [CLRS09, p. 435].

La corrección del algoritmo se sigue de los dos resultados anteriores, aplicando inducción.

Grafos



- Grafos etiquetados con pesos no negativos



- **Longitud** o coste de un camino: **Suma** de los pesos de las aristas que lo componen
- Características del tipo de dato de las etiquetas (o pesos):
 - Dominio ordenado ($<$, $=$, $\leq$)
 - Operación suma (+):
 - Conmutativa
 - Con elemento neutro (0), que es además el peso más pequeño posible
 - Con elemento idempotente (∞), que es además el peso más grande posible



Representaciones de grafos

Matriz de adyacencia

Grafo dirigido $G = (V, A)$ con $V = \{1, 2, \dots, n\}$.

La **matriz de adyacencia** para G es una matriz A de dimensiones $n \times n$ de elementos booleanos en la que $A[i, j]$ es verdad si y sólo si existe una arista en G que va del vértice i al vértice j .



$$A = \begin{bmatrix} \text{falso} & \text{verdad} & \text{falso} \\ \text{verdad} & \text{falso} & \text{verdad} \\ \text{falso} & \text{falso} & \text{falso} \end{bmatrix}$$

Matriz de adyacencia

- En el caso de un grafo no dirigido, la matriz de adyacencia tiene la particularidad de ser simétrica.



Representaciones de grafos

Matriz de adyacencia

- La representación de la matriz de adyacencia es útil para aquellos algoritmos que precisan saber **si existe o no** una arista entre dos vértices dados.
- En el caso de **grafos etiquetados** con pesos, los elementos de la matriz pueden ser enteros o reales (en lugar de booleanos) y representar el peso de la arista. Si no existe una arista de i a j , debe emplearse un valor que no pueda ser una etiqueta válida para almacenarse en $A[i, j]$.
- Un **inconveniente** de la representación de la matriz de adyacencia es que requiere un **espacio** en $\Theta(n^2)$ aunque el grafo tenga muy **pocas aristas**.
- Dado un grafo $G = (V, A)$ representado mediante su matriz de adyacencia, la pregunta $\iota(u, v) \in A?$ se puede contestar en un tiempo en $\Theta(1)$.
- Sin embargo, la operación $\text{sucesores}(g, v)$ necesita un tiempo en $\Theta(n)$ (hay que recorrer una fila completa de la matriz).
- Más aún, para saber, por ejemplo, si un grafo no dirigido representado mediante su matriz de adyacencia es conexo, o simplemente para conocer el número de aristas, los algoritmos requieren un tiempo en $\Theta(n^2)$, lo cual es más de lo que cabría esperar.



Representaciones de grafos

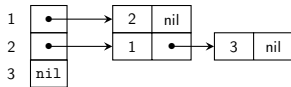
Listas de adyacencia

- Esta representación sirve para mejorar la eficiencia de los algoritmos sobre grafos que necesitan acceder a los vértices sucesores a uno dado.
- Consiste en almacenar n listas enlazadas mediante punteros de forma que la lista i -ésima contiene los vértices sucesores del vértice i .

```

1  constante n = ... {cardinal de V}
2  tipos ptNodo = ↑ nodo;
3      nodo = registro {
4          vértice:1..n;
5          sig:ptNodo
6      }
7  grafo = vector[1..n] de ptNodo
  
```

Grafo dirigido



Vector de listas de adyacencia



Representaciones de grafos

Listas de adyacencia

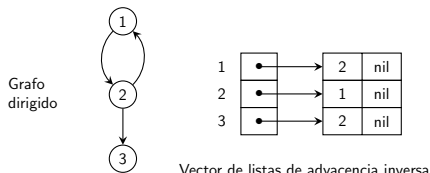
- La representación de un grafo con listas de adyacencia requiere un espacio del orden del máximo entre
 - n (el número de vértices) y
 - a (el número de aristas).
- En el caso de un grafo no dirigido, si $(u, v) \in A$ entonces el vértice v estará en la lista correspondiente al vértice u , y el vértice u lo estará a su vez en la lista de adyacencia del vértice v .
- Para representar un grafo etiquetado con pesos basta con añadir otro campo al tipo nodo que almacene el peso correspondiente.
- Acceder a la lista de los vértices sucesores de uno dado lleva un tiempo constante.
- Sin embargo, consultar si una determinada arista (u, v) está en el grafo precisa recorrer la lista asociada al vértice u , lo que en el caso peor se realiza en $\Theta(n)$.



Representaciones de grafos

Listas de adyacencia

- En el caso de un grafo dirigido, si se necesita saber los predecesores de un vértice dado, la representación con listas de adyacencia no es adecuada.
- Ese problema puede superarse si se almacena otro vector de n listas de forma que la lista i -ésima contenga los vértices predecesores del vértice i .
- Estas listas se llaman listas de adyacencia inversa.

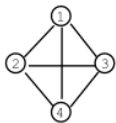




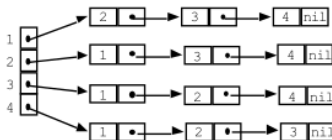
Representaciones de grafos

Listas múltiples de adyacencia

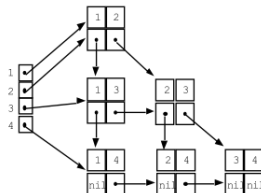
- En el caso de grafos dirigidos, las listas de adyacencia directa e inversa pueden representarse de forma más compacta. . .
- Una lista múltiple es una estructura dinámica de datos enlazados mediante punteros en la que cada dato puede pertenecer a dos o más listas.
- Hay un nodo por cada una de las aristas del grafo.
- Cada nodo guarda la clave de dos vértices (origen y destino de la arista) y dos punteros: el primero al nodo que guarda la siguiente arista que tiene el mismo vértice destino y el segundo al nodo que guarda la siguiente arista que tiene el mismo vértice origen.



Grafo no dirigido



Listas de adyacencia



Listas múltiples de adyacencia



Representaciones de grafos

Listas múltiples de adyacencia

- La definición de la estructura de datos es la siguiente:

```
1  constante n = ... {cardinal de V}
2  tipos ptNodo = ↑ nodo;
3      nodo = registro {
4          v1,v2:1..n;
5          sig1,sig2:ptNodo
6      }
7      grafoNoDirigido = vector[1..n] de ptNodo
```

- Una ventaja de esta representación es que es útil para implementar aquellos algoritmos que necesitan recorrer un grafo y al mismo tiempo “marcar” las aristas por las que se pasa (bastaría con añadir un campo booleano al tipo nodo).



Caminos mínimos en grafos

- Cálculo del coste del camino mínimo desde un vértice dado al resto, en un grafo etiquetado con pesos no negativos
- Utilidad:
El grafo representa una distribución geográfica, donde las aristas dan el coste (precio, distancia...) de la conexión entre dos lugares y se desea averiguar el camino más corto (barato...) para llegar a un punto partiendo de otro



Caminos mínimos en grafos

E.W. Dijkstra

“A note on two problems in connexion with graphs”^a, Numerical Mathematica, 1, pp. 269-271, 1959.

^a<https://www.cs.utexas.edu/users/EWD/ewd08xx/EWD841a.PDF>

EWD841a-01

Dijkstra E.W. A Note on Two Problems in Connexion with Graphs
Numerische Mathematik 1: 269-271, 1959
[Mathematisch Centrum, Amsterdam]

Abstract. We consider a graph with n vertices, all pairs of which are connected by an edge; each edge is of given positive length. The following two basic problems are solved.

Problem 1. Construct the tree of minimal total length between the n vertices. (A tree is a graph with one and only one path between any two vertices.)

Problem 2. Find the path of minimal total length between two given vertices.



Caminos mínimos en grafos

Solución voraz: Algoritmo de Dijkstra

- Para grafos dirigidos (la extensión a no dirigidos es inmediata)
- Genera uno a uno los caminos de un nodo v al resto por orden creciente de longitud
- Usa un conjunto de vértices donde, a cada paso, se guardan los nodos para los que ya se sabe el camino mínimo
- Devuelve un vector indexado por vértices: en cada posición w se guarda el coste del camino mínimo que conecta v con w
- Cada vez que se incorpora un nodo a la solución se comprueba si los caminos todavía no definitivos se pueden acortar pasando por él
- Se supone que el camino mínimo de un nodo a sí mismo tiene coste nulo
- Valor ∞ en la posición w del vector indica que no hay ningún camino desde v a w .



Caminos mínimos en grafos

```

1  // Pre: g es un grafo dirigido etiquetado no neg.
2  funcion Dijkstra(g:grafo; v:vért) devuelve vector[vért] de etiq
3  variables S:cjtVért; D:vector[vért] de etiq
4  principio
5       $\forall w \in \text{vért}: D[w] := \text{etiqueta}(g, v, w);$ 
6       $D[v] := 0; S := \{v\};$ 
7      mientras que S no contenga todos los vértices {
8          // D contiene caminos mínimos formados íntegramente por nodos de S
9          // (excepto el último), y los nodos de S corresponden a los caminos
10         // mínimos más cortos calculados hasta el momento
11         elegir  $w \notin S$  t.q.  $D[w]$  es mínimo;
12          $S := S \cup \{w\};$ 
13          $\forall u \notin S$ : actualizar dist.mín. comprobando si por w hay un atajo
14     }
15     devuelve D
16 fin
17 // Post:  $D = \text{caminosMínimos}(g, v)$ 

```



Caminos mínimos en grafos

- Corrección: ver material adicional en la web.
- Se presenta a continuación una implementación más detallada
 - Se utiliza en lugar de S su complementario T
 - Se supone que n es el número de vértices



Caminos mínimos en grafos

```

1  // Pre: g es un grafo dirigido etiquetado no neg.
2  funcion Dijkstra(g:grafo; v:vért) devuelve vector[vért] de etiq
3  variables T:cjtVért;
4          D:vector[vért] de etiq;
5          u,w:vért; val:etiq
6  {
7      T:=∅;
8      para todo w en vért {
9          D[w]:=etiqueta(g,v,w); T:=T∪{w}
10     };
11     D[v]:=0; T:=T\{v};
12     repetir n-2 veces { // quedan n-1 caminos por determinar
13         // selección del w tal que D[w] es mín.:  $w \in T \wedge \forall u \in T: D[w] \leq D[u]$ 
14         val:=∞;
15         para todo u en T {
16             si  $D[u] \leq val$  { w:=u; val:=D[u] }
17             // siempre hay un nodo que cumple la condición
18         };
19     ...

```



Caminos mínimos en grafos

```

1      ...
2      // se marca w como vértice tratado
3      T:=T-{w};
4      // se recalculan las nuevas dist. mínimas
5      para todo u en T {
6          si D[w]+etiqueta(g,w,u)<D[u] {
7              D[u]:=D[w]+etiqueta(g,w,u)
8          }
9      }
10     };
11     devuelve D
12 }
13 // Post: D=caminosMínimos(g,v)

```

Nota: el bucle principal se ejecuta $n - 2$ veces porque el último camino queda calculado después del último paso (no quedan vértices para hallar atajos)



Caminos mínimos en grafos. Tiempo de ejecución

- se supone que las operaciones sobre conjuntos están implementadas en tiempo constante, excepto la creación (p.ej., mediante un vector de booleanos)
- fase de inicialización:
 - creación del conjunto y ejecución n veces de diversas operaciones constantes: $\Theta(n)$
- fase de selección:
 - las instrucciones del interior del bucle son $\Theta(1)$
 - número de ejecuciones del bucle:
 - 1ª vuelta: se consultan $n - 1$ vértices,
 - 2ª vuelta: $n - 2$, etc. (el cardinal de T decrece en 1 en cada paso)
 - n^{o} de ejecuciones: $n(n - 1)/2 - 1 \implies \Theta(n^2)$
- fase de “marcaje”:
 - n supresiones a lo largo del algoritmo: $\Theta(n)$
- fase de recálculo de las distancias mínimas:
 - queda $\Theta(n^2)$ por igual razón que la selección

Coste total: $\Theta(n^2)$



Caminos mínimos en grafos. Tiempo de ejecución

Algunas mejoras.

- Si la representación es por listas de adyacencia, la fase de recálculo se ejecuta sólo a $(a < n^2)$ veces (sustituyendo el bucle sobre los vért. de T por otro bucle sobre los vért. sucesores de w).
- Si el conjunto T se sustituye con una cola con prioridades, se rebaja también el coste de la fase de selección (puesto que se selecciona el mínimo).

Problema: la fase de recálculo puede exigir cambiar la prioridad de un elemento cualquiera de la cola.

Solución: nuevo tipo de cola con prioridades que permita el acceso directo a cualquier elemento.



Caminos mínimos en grafos

```

1  // Pre: g es un grafo dirigido etiquetado no neg.
2  funcion Dijkstra(g:grafo; v:vért) devuelve vector[vért] de etiq
3  variables A:cpa; // cola de aristas con prior.
4          D:vector[vért] de etiq; u,w:vért; et,val:etiq
5  {
6      // crear la cola con todas las aristas <v,w>
7      creaVacía(A);
8      para todo w en vért { inserta(A, w, etiqueta(g, v, w)) };
9      mientras que not esVacía(A) {
10         <w,val>:=primero(A); // selecc. arista mín.
11         D[w]:=val; borra(A); // marca w como tratado
12         // se recalculan las nuevas dist. mínimas
13         para todo <u,et> en suc(g, w) {
14             si está(A, u) y_entonces val + et < valor(A, u) {
15                 sustituye(A, u, val + et) }
16         }
17     };
18     D[v]:=0; devuelve D
19 } // Post: D=caminosMínimos(g,v)

```



Caminos mínimos en grafos

- Eficiencia temporal:
 - inicialización: $\Theta(n \log(n))$
 - selección y supresión: $\Theta(n \log(n))$
 - bucle interno: examina todas las aristas del grafo y en el caso peor efectúa una sustitución por arista, por tanto: $\Theta(a \log(n))$
- El coste total es: $\Theta((a + n) \log(n))$, luego es mejor que la versión anterior si el grafo es disperso.
- Si el grafo es denso, el algoritmo original es mejor.
- Puede conseguirse un coste en $\Theta(n^2)$ para grafos densos y en $\Theta(a \log(n))$ para no densos, representando el montículo en un árbol k-ario en lugar de binario [BB97, pp. 243].



Caminos mínimos en grafos. Comparación.

Fase	Matriz	Lista
Inicialización	$\Theta(n)$	$\Theta(n \log(n))$
Selección y supresión	$\Theta(n)$	$\Theta(n \log(n))$
Recálculo/sustitución	$\Theta(n^2)$	$\Theta(a \log(n))$
Total	$\Theta(n^2)$	$\Theta((a + n) \log(n))$

En principio ...

$$n \approx a \ll n^2$$



Caminos mínimos en grafos

Ejercicio: cálculo de la secuencia de nodos que componen el camino mínimo

- si el camino mínimo entre v y w pasa por u , el camino mínimo entre v y u es un prefijo del camino mínimo entre v y w
- basta con devolver un vector $pred$ tal que $pred[w]$ contenga el nodo predecesor de w en el camino mínimo de v a w (que será v si está directamente unido al nodo de partida o si no hay camino entre v y w)
- el vector debe actualizarse al encontrarse un atajo en el camino
- es necesario también diseñar un nuevo algoritmo para recuperar el camino a un nodo dado, que tendría como parámetro $pred$

Árboles de recubrimiento de coste mínimo



- **Objetivo:** dado un grafo, obtener un nuevo grafo que sólo contenga las aristas imprescindibles para una **optimización global** de las conexiones entre todos los nodos



Árboles de recubrimiento de coste mínimo

- **Objetivo:** dado un grafo, obtener un nuevo grafo que sólo contenga las aristas imprescindibles para una **optimización global** de las conexiones entre todos los nodos

Global

Algunos pares de nodos pueden no quedar conectados entre ellos con el mínimo coste posible en el grafo original.



Árboles de recubrimiento de coste mínimo

- **Objetivo:** dado un grafo, obtener un nuevo grafo que sólo contenga las aristas imprescindibles para una **optimización global** de las conexiones entre todos los nodos
- **Aplicación:** problemas que tienen que ver con distribuciones geográficas:
 - Conjunto de computadores distribuidos geográficamente en diversas ciudades de diferentes países a los que se quiere conectar para intercambiar datos, compartir recursos, etc.;
 - Se pide a las compañías telefónicas respectivas los precios de alquiler de líneas entre ciudades
 - Asegurar que todos los computadores pueden comunicar entre sí, minimizando el precio total de la red



Terminología

- Un **árbol libre** es un grafo no dirigido conexo acíclico
 - Todo árbol libre con n vértices tiene $n-1$ aristas
 - Si se añade una arista se introduce un ciclo
 - Si se borra una arista quedan vértices no conectados
 - Cualquier par de vértices está unido por un único camino simple
 - Un árbol libre con un vértice distinguido es un árbol con raíz
- **Árbol de recubrimiento** de un grafo no dirigido y etiquetado no negativamente: es cualquier subgrafo que contenga todos los vértices y que sea un árbol libre
- **Árbol de recubrimiento de coste mínimo:** es un árbol de recubrimiento y no hay ningún otro árbol de recubrimiento cuya suma de aristas sea menor
- Notación:

$$arm(g) = \{\text{árboles de recubrimiento de coste mínimo de } g\}$$



Árboles de recubrimiento de coste mínimo

Propiedad fundamental

Sea g un grafo no dirigido conexo y etiquetado no negativamente, y U un conjunto de vértices,

$$g \in \{f : V \times V \rightarrow E\}, U \subset V, U \neq \emptyset$$

Si $\langle u, v \rangle$ es la arista más pequeña de g tal que $u \in U$ y $v \in V \setminus U$, entonces existe algún árbol de recubrimiento de coste mínimo de g que la contiene.

Dem.: Por reducción al absurdo.

(por ejemplo, en

[AHU88] A.V. Aho, J.E. Hopcroft y J.D. Ullman. Estructuras de datos y algoritmos. Addison-Wesley Iberoamericana, S.A., 1988).



Árboles de recubrimiento de coste mínimo

V. Jarník

“O jistém problému minimálním”, Práce Moravské Přírodovědecké Společnosti, 6, pp. 57-63, 1930.

R.C. Prim

“Shortest connection networks and some generalizations”, Bell System Technical Journal, 36, pp. 1389-1401, 1957.

Algoritmo de Prim (debido a Jarník)

- Aplica reiteradamente la propiedad de los árboles de recubrimiento de coste mínimo incorporando a cada paso una arista
- Se usa un conjunto U de vértices tratados y se selecciona en cada paso la arista mínima que une un vértice de U con otro de su complementario



Árboles de recubrimiento de coste mínimo

```

1  // Pre: g es un grafo no dirigido conexo etiquetado no negativamente
2  funcion Prim(g:grafo) devuelve grafo
3  variables U:cjtVért; gsol:grafo;
4          u,v:vért; x:etiq
5  {
6      creaVacío(gsol); U:={cualquier vértice};
7      mientras que U no contenga todos los vért. {
8          seleccionar <u,v,x> mínima t.q. u∈U; v∉U
9          añade(gsol,u,v,x); U:=U∪{v}
10     }
11     devuelve gsol
12     }
13  // Post: gsol ∈ arm(g)

```

Coste: $\Theta(na)$
 (es decir, $\Theta(n^3)$ si el grafo es denso)



Árboles de recubrimiento de coste mínimo

La versión previa puede refinarse hasta obtener un algoritmo en $\Theta(n^2)$, es decir, mejor que el anterior ($a \geq n - 1$).

Se usa un vector *arisMin*, indexado por vértices, que contiene:

- Si $v \notin U$: $arisMin[v] = \langle w, g(v, w) \rangle$ t.q. $\langle v, w \rangle$ es la arista más pequeña que conecta v con un vértice $w \in U$
- Si $v \in U$: $arisMin[v] = \langle v, \infty \rangle$



Árboles de recubrimiento de coste mínimo

```

1  // Pre: g es un grafo no dirigido conexo etiquetado no negativamente
2  funcion Prim(g:grafo) devuelve grafo
3  variables
4    arisMín:vector[vért] de <vért,etiq>;
5    gsol:grafo; prim,mín,v,w:vért; x:etiq
6  { // inicialización del resultado
7    prim := unVérticeCualquiera;
8    para todo v en vért {
9      arisMín[v] := <prim, etiqueta(g,prim,v)>
10   }
11   arisMín[prim] := <prim, ∞>;
12
13   // a continuación se aplica el método
14   creaVacío(gsol);
15   repetir n-1 veces {
16     mín := prim; // centinela: arisMín[mín].et=∞
17     para todo v en vért {
18       <w,x> := arisMín[v];
19       si x < arisMín[mín].et { mín := v // como mínimo habrá uno }
20     }

```



Árboles de recubrimiento de coste mínimo

```

1
2  // a continuación se añade a la solución
3  añade(gsol, mín, arisMín[mín].v, arisMín[mín].et);
4
5  // se añade mín al conjunto de vértices tratados
6  arisMín[mín] := <mín, ∞>;
7
8  // se reorganiza el vector comprobando si la arista mínima de los
9  // vértices todavía no tratados los conecta a mín
10 para todo <v,x> en adyacentes(g,mín) {
11     si (arisMín[v].v ≠ v) and (x < arisMín[v].et) {
12         arisMín[v] := <mín, x>
13     }
14 }
15 }
16 devuelve gsol
17 }
18 // Post: gsol ∈ arm(g)

```



Eficiencia temporal

- Inicialización:
 - Lineal en caso de matriz de adyacencia y
 - Cuadrática en caso de listas
- Bucle principal:
 - el bucle de selección: $\Theta(n)$
 - el añadido de una arista al grafo:
 - Constante usando matriz,
 - Lineal usando listas
- El bucle de reorganización:
 - con matriz de adyacencia: el cálculo de los adyacentes es $\Theta(n)$ y el coste total queda $\Theta(n^2)$
 - con listas: el coste total es $\Theta(a + n)$

Coste total: $\Theta(n^2)$, independientemente de la representación.

Coste espacial: $\Theta(n)$ de espacio adicional.



Árboles de recubrimiento de coste mínimo

J.B. Kruskal

“On the shortest spanning subtree of a graph and the traveling salesman problem”, Proceedings of the American Mathematical Society, 7, pp. 48-50, 1956.

Algoritmo de Kruskal:

- Se basa también en la propiedad de los árboles de recubrimiento de coste mínimo:

Partiendo del árbol vacío, se selecciona en cada paso la arista de menor etiqueta que no provoque ciclo sin requerir ninguna otra condición sobre sus extremos.



Árboles de recubrimiento de coste mínimo

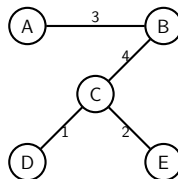
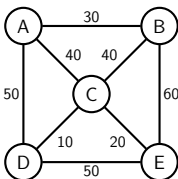
```
1 // Pre: g es un grafo no dirigido conexo etiquetado no negativamente
2 función Kruskal(g:grafo) devuelve grafo
3 variables gsol:grafo; u,v:vért; x:eti
4 {
5     creaVacío(gsol);
6     mientras que gsol no contenga todos los vértices hacer {
7         seleccionar <u,v,x> mínima(*) no examinada;
8         si no provoca ciclo {
9             añade(gsol,u,v,x)
10        }
11    }
12    devuelve gsol
13 }
14 // Post: gsol ∈ arm(g)
```

(*) Utilizar una cola con prioridades.

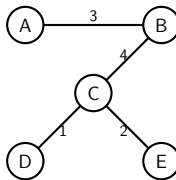
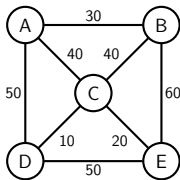


Implementación eficiente

- En cada momento, los vértices que están dentro de una componente conexa en la solución forman una clase de equivalencia, y el algoritmo se puede considerar como la fusión continuada de clases hasta obtener una única componente con todos los vértices del grafo.



Implementación eficiente

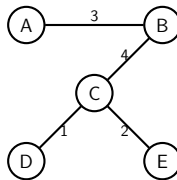
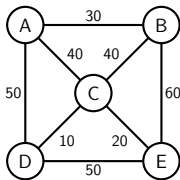


Evolución de las clases de equivalencia:

$$\{[A],[B],[C],[D],[E]\}$$



Implementación eficiente

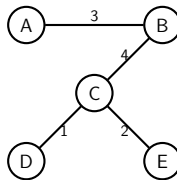
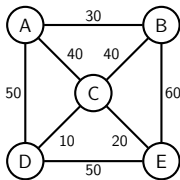


Evolución de las clases de equivalencia:

$$\{[A],[B],[C],[D],[E]\} \rightarrow \{[A],[B],[C,D],[E]\}$$



Implementación eficiente

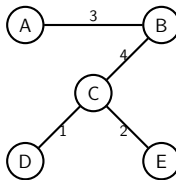
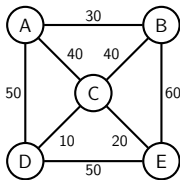


Evolución de las clases de equivalencia:

$$\{[A],[B],[C],[D],[E]\} \rightarrow \{[A],[B],[C,D],[E]\} \rightarrow \{[A],[B],[C,D,E]\}$$



Implementación eficiente

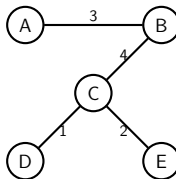
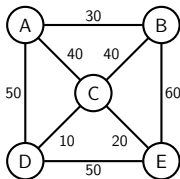


Evolución de las clases de equivalencia:

$$\{[A],[B],[C],[D],[E]\} \rightarrow \{[A],[B],[C,D],[E]\} \rightarrow \{[A],[B],[C,D,E]\} \rightarrow \{[A,B],[C,D,E]\}$$



Implementación eficiente



Evolución de las clases de equivalencia:

$$\{[A],[B],[C],[D],[E]\} \rightarrow \{[A],[B],[C,D],[E]\} \rightarrow \{[A],[B],[C,D,E]\} \rightarrow \{[A,B],[C,D,E]\} \rightarrow \{[A,B,C,D,E]\}$$



Árboles de recubrimiento de coste mínimo

Se utiliza el TAD “relación de equivalencia” sobre los vértices

```

1  genero rev {relac. de equiv. sobre vért.}
2  operaciones
3    creaREV: -> rev {cada vért. una clase}
4    clase: rev vért -> nat
5    fusiona: rev nat nat -> rev
6    numClases: rev -> nat

```

Implementación asintóticamente óptima, basada en una representación del tipo rev usando árboles y una técnica de compresión de caminos [AHU88, pp. 182-191] (o ver aquí: <http://webdiis.unizar.es/asignaturas/TAP/material/3.5.disjuntos.pdf>)

- El coste de *creaREV* es lineal
- El coste de *numClases* es constante
- El coste de k ejecuciones combinadas de *fusiona* y *clase* es $\Theta(k\alpha(k, n))$, lo cual es prácticamente constante, porque α es una función inversa de la función de Ackerman que crece MUY despacio ($\alpha(k, n) \leq 4$, para todos los valores de k y n “imaginables”)



Árboles de recubrimiento de coste mínimo

```
1 // Pre: g es un grafo no dirigido conexo etiquetado no negativamente
2 funcion Kruskal(g:grafo) devuelve grafo
3 variables T:cpa; gsol:grafo;
4         u,v:vért; x:etiq;
5         C:rev; ucomp,vcomp:nat
6 {
7     creaREV(C); // cada vért. forma una clase
8     creaVacío(gsol);
9     creaVacía(T);
10
11     // se colocan todas las aristas en la cola
12     para todo v en vért {
13         para todo <u,x> en adyacentes(g,v) {
14             inserta(T,v,u,x)
15         }
16     }
17
18     ...
```



Árboles de recubrimiento de coste mínimo

```
1  ...
2
3  mientras que numClases(C) > 1 {
4      // obtener y elim. la arista mín.de la cola
5      <u,v,x> := primero(T); borra(T);
6
7      // si la arista no provoca ciclo se añade a la
8      // solución y se fusionan las clases corresp.
9      ucomp := clase(C,u); vcomp := clase(C,v);
10
11     si ucomp ≠ vcomp {
12         fusiona(C, ucomp, vcomp);
13         añade(gsol, u, v, x)
14     }
15 }
16 devuelve gsol
17 }
18 // Post: gsol ∈ arm(g)
```




Árboles de recubrimiento de coste mínimo

Coste del algoritmo:

- las a inserciones consecutivas de aristas en la cola con prioridades dan $\Theta(a \log(a))$; como $a \leq n^2$: $\Theta(a \log(a)) \subseteq \Theta(a \log(n^2)) = \Theta(2a \log(n)) = \Theta(a \log(n))$
- como mucho hay a consultas y supresiones de la arista mínima, el coste de la consulta es constante y el de la supresión es $\Theta(\log(a))$; por ello, este paso queda en $\Theta(a \log(n))$
- averiguar cuántas clases hay en la relación de equivalencia es constante
- en el caso peor, la operación de fusión de clases se ejecuta $n - 1$ veces y la operación de localizar la clase $2a$ veces; por tanto, el coste total es en la práctica $\Theta(a)$
- las $n - 1$ inserciones de aristas quedan en $\Theta(n)$ con matriz de adyacencia y $\Theta(n^2)$ con listas, aunque en el caso de las listas puede reducirse también a $\Theta(n)$ si se elimina en la operación de añade la comprobación de existencia de la arista (el algoritmo de Kruskal garantiza que no habrá inserciones repetidas de aristas)

Coste total: $\Theta(a \log(n))$ (menos que el algoritmo de Prim, aunque con mayor espacio adicional)



El problema de la selección de actividades

- Es un problema de **planificación de tareas** (en inglés, *scheduling*).
- Se tiene un conjunto $S = \{1, 2, \dots, n\}$ de **actividades** (por ej., clases) que deben usar un **recurso** (por ej., un aula) que sólo puede ser usado por **una actividad en cada instante**.
- Cada actividad i tiene asociado un **instante de comienzo**, un **instante de finalización**:

$$c_i, f_i, (c_i \leq f_i),$$

de forma que la actividad i , si se realiza, debe hacerse durante $[c_i, f_i)$.

- Dos actividades i, j se dicen **compatibles** si los intervalos $[c_i, f_i)$ y $[c_j, f_j)$ no se superponen

$$(c_i \geq f_j) \vee (c_j \geq f_i)$$



El problema de la selección de actividades

El problema de selección de actividades consiste en elegir un **conjunto de actividades mutuamente compatibles que tenga cardinal máximo**.

$$\begin{aligned} S &= \{1, 2, \dots, n\} \\ \forall i \in \{1, \dots, n\}, \\ &c_i, f_i, (c_i \leq f_j), \\ &(c_i \geq f_j) \vee (c_j \geq f_i) \end{aligned}$$



Ejemplos de estrategias voraces

Funciones de selección voraz para resolver el problema:

Seleccionar primero la tarea que ...

- empieza primero.
- termina primero.
- es más corta.
- es menos conflictiva.



El problema de la selección de actividades

```
1  constante n = ... // número de actividades
2  tipo vectReal = vector[1..n] de real
3
4  // Pre:  $\forall i \in 1..n : c[i] \leq f[i] \wedge \forall i \in 1..n - 1 : f[i] \leq f[i + 1]$ 
5  funcion selec(c,f:vectReal) devuelve conjunto
6  variables i,j:entero; A:conjunto
7  {
8    A := {1};
9    j := 1; // j es la última actividad seleccionada
10   para i:=2 hasta n hacer {
11     si c[i]  $\geq$  f[j] {
12       A := A  $\cup$  {i}; j := i
13     }
14   }
15   devuelve A
16 }
17 // Post: A es soluc. óptima del problema de la selección de actividades.
```



El problema de la selección de actividades

```

1  constante n = ... // número de actividades
2  tipo vectReal = vector[1..n] de real
3
4  // Pre:  $\forall i \in 1..n : c[i] \leq f[i] \wedge \forall i \in 1..n - 1 : f[i] \leq f[i + 1]$ 
5  funcion selec(c,f:vectReal) devuelve conjunto
6  variables i,j:entero; A:conjunto
7  {
8    A := {1};
9    j := 1; // j es la última actividad seleccionada
10   para i:=2 hasta n hacer {
11     si c[i]  $\geq$  f[j] {
12       A := A  $\cup$  {i}; j := i
13     }
14   }
15   devuelve A
16 }
17 // Post: A es soluc. óptima del problema de la selección de actividades.

```

Coste temporal: $\Theta(n)$



Demostración de la corrección

La actividad 1 tiene el primer instante de finalización. Veamos que existe una solución óptima que comienza con una selección voraz, es decir, con la actividad 1.

Supongamos que $A \subseteq S$ es una solución óptima y que las actividades de A están ordenadas por tiempos de finalización.

Supongamos que la primera actividad de A es la actividad k .

- Si $k = 1$, entonces A empieza con una selección voraz.
- Si $k \neq 1$, veamos que existe otra solución óptima B que comienza con la actividad 1.
 - Sea $B = A \setminus \{k\} \cup \{1\}$.
Como $f_1 \leq f_k$, las actividades en B son disjuntas y como tiene el mismo número de actividades que A entonces B es también óptima.

(Sigue...)



Demostración de la corrección. Continuación.

Después de la selección voraz de la actividad 1, el problema se reduce a encontrar una solución óptima del problema de selección actividades S' sobre las actividades de S que son compatibles con la actividad 1.

Si A' es una solución óptima para el problema $S' = \{i \in S \mid c_i \geq f_1\}$, entonces $A = A' \cup \{1\}$ es una solución óptima para el problema original S .

- Trivial, si existiese alguna solución mejor que A para S , las actividades de esa solución eliminando la tarea 1 serían una solución mejor para S' que A' .

Por tanto, después de cada selección voraz, nos encontramos con otro problema de optimización con igual forma que el problema original (subestructura óptima).

Por inducción sobre el número de selecciones, y realizando una selección voraz en cada paso, se obtiene una solución óptima. □



El problema de la minimización del tiempo de espera

Otro problema de planificación de tareas

- Un servidor tiene que atender a n clientes que llegan todos juntos al sistema.
- El tiempo de servicio para cada cliente es $t_i, i = 1, 2, \dots, n$.
- Se quiere minimizar:

$$T = \sum_{i=1}^n \text{tiempo total que el cliente } i \text{ está en el sistema}$$



El problema de la minimización del tiempo de espera

Otro problema de planificación de tareas

- Un servidor tiene que atender a n clientes que llegan todos juntos al sistema.
- El tiempo de servicio para cada cliente es $t_i, i = 1, 2, \dots, n$.
- Se quiere minimizar:

$$T = \sum_{i=1}^n \text{tiempo total que el cliente } i \text{ est\'a en el sistema}$$

Ejemplos

un procesador, un cajero autom\'atico, un surtidor de gasolina, etc.)





Ejemplo

3 clientes con $t_1 = 5$, $t_2 = 10$, $t_3 = 3$.

orden	T					
1 2 3:	5	+	(5+10)	+	(5+10+3)	= 38
1 3 2:	5	+	(5+3)	+	(5+3+10)	= 31
2 1 3:	10	+	(10+5)	+	(10+5+3)	= 43
2 3 1:	10	+	(10+3)	+	(10+3+5)	= 41
3 1 2:	3	+	(3+5)	+	(3+5+10)	= 29 ← óptimo
3 2 1:	3	+	(3+10)	+	(3+10+5)	= 34



Ejemplo

3 clientes con $t_1 = 5$, $t_2 = 10$, $t_3 = 3$.

orden	T					
1 2 3:	5	+	(5+10)	+	(5+10+3)	= 38
1 3 2:	5	+	(5+3)	+	(5+3+10)	= 31
2 1 3:	10	+	(10+5)	+	(10+5+3)	= 43
2 3 1:	10	+	(10+3)	+	(10+3+5)	= 41
3 1 2:	3	+	(3+5)	+	(3+5+10)	= 29 ← óptimo
3 2 1:	3	+	(3+10)	+	(3+10+5)	= 34

Estrategia voraz: Atender en cada paso al cliente no atendido con menor tiempo de servicio.



Demostración de la corrección:

Sea $I = (i_1, i_2, \dots, i_n)$ una permutación cualquiera de los naturales $\{1, 2, \dots, n\}$.

Si se atiende a los clientes según la secuencia I , el tiempo total que los clientes están en el sistema es:

$$\begin{aligned} T(I) &= t_{i_1} + (t_{i_1} + t_{i_2}) + ((t_{i_1} + t_{i_2}) + t_{i_3}) + \dots \\ &= nt_{i_1} + (n-1)t_{i_2} + (n-2)t_{i_3} + \dots \\ &= \sum_{k=1}^n (n-k+1)t_{i_k} \end{aligned}$$

Supongamos que I es tal que es posible encontrar dos naturales a y b con

$$a < b \text{ y } t_{i_a} > t_{i_b},$$

es decir, que el cliente a -ésimo es atendido antes que el b -ésimo aunque su tiempo de servicio es mayor.

Si se invierten sus posiciones en I se obtiene una nueva secuencia I' .

(Sigue...)



Continuación

Entonces:

$$\begin{aligned}
 T(I') &= (n - a + 1)t_{i_b} + (n - b + 1)t_{i_a} + \sum_{k=1, k \neq a, b}^n (n - k + 1)t_{i_k} \\
 T(I) - T(I') &= (n - a + 1)(t_{i_a} - t_{i_b}) + (n - b + 1)(t_{i_b} - t_{i_a}) \\
 &= (b - a)(t_{i_a} - t_{i_b}) \\
 &> 0
 \end{aligned}$$

Es decir, se puede mejorar cualquier secuencia en la que un cliente que necesita más tiempo sea atendido antes que otro que necesita menos. Las únicas secuencias que no pueden mejorarse, y por tanto son óptimas, son aquellas en las que los clientes están ordenados por tiempos de servicio decrecientes (todas ellas dan igual tiempo total). $\square$

- Puede generalizarse a s procesadores.
 - Se numeran los clientes de forma que $t_1 \leq t_2 \leq \dots \leq t_n$.
 - El procesador i , $1 \leq i \leq s$, atenderá a los clientes $i, i + s, i + 2s, \dots$



Fundamentos teóricos del esquema voraz

- Existe una teoría matemática que sirve de base a los algoritmos voraces:

Teoría de matroides

Definición

- Un matroide es un par $M = (S, I)$ tal que:
 - S es un conjunto finito no vacío.
 - I es una familia no vacía de subconjuntos de S , llamados subconjuntos **independientes**, tal que si $B \in I$ y $A \subseteq B$, entonces $A \in I$.
Por cumplir esta propiedad, I se dice **hereditaria** (nótese que el conjunto vacío es necesariamente un miembro de I).
 - Si $A \in I$, $B \in I$ y $|A| < |B|$, entonces existe algún $x \in B \setminus A$ tal que $A \cup \{x\} \in I$.
Se dice que M satisface la **propiedad de intercambio**.



Ejemplos de matroides

Matroide matricial (H. Whitney, 1935)

Dada una matriz de números reales, sea S el conjunto de sus vectores fila, y considérese que un subconjunto de S es independiente si los vectores que lo componen son linealmente independientes (en el sentido habitual).

Matroide gráfico

- Sea un grafo no dirigido $G = (V, A)$.

Sea $M_G = (S_G, I_G)$, donde:

- $S_G = A$ (el conjunto de aristas).
- Si $B \subseteq A$, entonces $B \in I_G$ si y sólo si B es acíclico.

→ muy relacionado con el cálculo de árboles de recubrimiento de coste mínimo.



Fundamentos teóricos del esquema voraz

Definición

Sea $M = (S, I)$ un matroide y $B \in I$.

Un elemento $x \notin B$ se dice **extensión** de B si $B \cup \{x\} \in I$.

Ejemplo

En un matroide gráfico M_G , si B es un conjunto independiente de aristas, entonces la arista a es una extensión de B si y sólo si a no está en B y la incorporación de a a B no crea un ciclo.



Fundamentos teóricos del esquema voraz

Definición

Si A es un subconjunto independiente de un matroide M , se dice que A es **maximal** si no tiene extensiones.

Teorema

Todos los subconjuntos independientes maximales de un matroide tienen el mismo cardinal.

(Dem.: muy fácil [CLRS09, p. 439].)

Ejemplo

Ejemplo: Dado un grafo no dirigido y conexo G , todo subconjunto independiente maximal de M_G es un árbol libre con $|V| - 1$ aristas que conecta todos los vértices de G .

Ese árbol se llama **árbol de recubrimiento** de G .



Fundamentos teóricos del esquema voraz

Definición

Un matroide $M = (S, I)$ al que se le asocia una función que asigna un “peso” estrictamente positivo $w(x)$ a cada $x \in S$ se llama matroide **ponderado**.

La función de peso w se extiende a los subconjuntos de S mediante la suma:

$$w(A) = \sum_{x \in A} w(x), \text{ para todo } A \subseteq S.$$

Ejemplo

Si $w(a)$ denota la longitud de la arista a en un matroide gráfico M_G , entonces $w(B)$ es la longitud total de todas las aristas del subconjunto de aristas B .



Voracidad y matroides

Fundamentos teóricos del esquema voraz

Muchos de los problemas para los que un algoritmo voraz calcula la solución óptima pueden formularse en términos de “encontrar un subconjunto independiente de máximo peso en un matroide ponderado.”

- Es decir, dado un matroide ponderado $M = (S, I)$, se quiere encontrar un subconjunto A independiente (i.e., $A \in I$) tal que $w(A)$ sea máximo.
- Un subconjunto independiente y de máximo peso en un matroide ponderado se llama subconjunto óptimo del matroide.
- Como la función de peso w es positiva, un subconjunto óptimo siempre es maximal (i.e., no tiene extensiones, ya que aumentar un conjunto siempre incrementa su peso).



Ejemplo: árbol de recubrimiento de coste mínimo

- Sea un grafo no dirigido y conexo $G = (V, A)$ y una función de longitud w tal que $w(a)$ es la longitud positiva de la arista a (llamamos longitudes a las etiquetas del grafo en lugar de pesos por reservar esta palabra para la función definida sobre un matroide).
- El problema consiste en encontrar un subconjunto de aristas que conecte todos los vértices del grafo y tenga longitud mínima.
- Considerar M_G y la función de peso w' definida como: $w'(a) = w_0 - w(a)$, donde w_0 es un valor mayor que la longitud de todas las aristas.
- En este matroide ponderado, todos los pesos son positivos y un subconjunto óptimo es precisamente un árbol de recubrimiento del grafo original con longitud total mínima.
- Más específicamente, cada subconjunto independiente maximal B corresponde a un árbol de recubrimiento, y puesto que

$$w'(B) = (|V|-1)w_0 - w(B)$$

para todo subconjunto independiente maximal B , el subconjunto independiente que maximiza $w'(B)$ debe minimizar $w(B)$.



Algoritmo voraz de cálculo de un subconjunto óptimo en un matroide arbitrario

```
1  // Pre: S y la función indep definen un matroide;
2  //      w es una función de peso positiva sobre S
3  funcion voraz(S:conjunto) devuelve conjunto
4  variable A:conjunto
5  {
6    A :=  $\emptyset$ ;
7    ordenar S por valores decrecientes del peso w;
8
9    para todo x en S tomado en orden decreciente de w(x) hacer {
10      si indep(A  $\cup$  {x}) {
11        A := A  $\cup$  {x}
12      }
13    }
14    devuelve A
15  }
16  // Post: A es subconjunto óptimo de (S,indep,w)
```



Fundamentos teóricos del esquema voraz

- Eficiencia temporal:

Denotando $|S|$ como n ,

- fase de ordenación: $\Theta(n \log n)$
- bucle: $\Theta(nf(n))$, suponiendo que el coste de evaluar la función indep es $\Theta(f(n))$.

Total: $\Theta(n \log n + nf(n))$

- Corrección:

Teorema 16.11 en [CLRS09, pp. 441-442] (un par de páginas; sin gran dificultad).



Voracidad y matroides

Utilidad fundamental

Dados un **problema** y una **solución voraz**,
para demostrar la **corrección** de esa solución basta con
identificar una estructura de **matroide subyacente** al problema



Voracidad y matroides

Resumen

- La teoría de matroides **no cubre** todos los problemas en los que el esquema voraz calcula la solución óptima.
- Sin embargo cubre muchos casos interesantes (por ej., árboles de recubrimiento de coste mínimo).
- Más aún, se ha demostrado que una estructura es un matroide si y sólo si el esquema voraz calcula para ella una solución óptima sea cual sea la función de peso escogida
(la demostración puede verse en pp. 13-18 de [Koz92] D.C. Kozen. The Design and Analysis of Algorithms. Springer- Verlag, 1992.)
- La investigación continúa...



Un problema de planificación de tareas a plazo fijo

- Se tiene un conjunto $S = \{1, 2, \dots, n\}$ de tareas de duración unidad.
- En cada instante, sólo se puede realizar una tarea.
- La tarea i , $1 \leq i \leq n$, debe terminarse antes del instante (o **plazo**) d_i , con $1 \leq d_i \leq n$.
- Hay una **penalización** w_i , $1 \leq i \leq n$, si la tarea i no termina antes de su plazo.
- Se trata de encontrar una secuencia de realización de las tareas que minimice la penalización total.



- Una tarea en una secuencia se dice **puntual** si termina antes de su plazo, y **tardía** si lo hace después de su plazo.
- Una secuencia está en **forma “puntuales primero”** si todas las tareas puntuales están antes de las tardías.
- Cualquier secuencia puede transformarse en forma puntuales primero (manteniendo la penalización).
En efecto, si una tarea puntual x va detrás de una tarea tardía y , pueden intercambiarse sus posiciones de forma que x sigue siendo puntual e y tardía.
- Una secuencia está en **forma canónica** si está en forma puntuales primero y las tareas puntuales están ordenadas por valores crecientes de sus plazos.
- Cualquier secuencia puede transformarse en forma canónica (manteniendo la penalización).
Argumentación similar a la anterior.



- Encontrar una secuencia óptima es encontrar un subconjunto A de tareas que sean puntuales en la secuencia óptima.
- Una vez encontrado A , la secuencia se crea ordenando los elementos de A por plazos crecientes y a continuación las tareas tardías (en $S \setminus A$) en cualquier orden.
- Decimos que un subconjunto A de tareas es independiente si existe una ordenación de las mismas en la que ninguna es tardía.
Por ejemplo, el conjunto de tareas puntuales de una secuencia es un conjunto independiente.
- Denotamos con I el conjunto de todos los subconjuntos independientes de tareas.



Sea $N_t(A)$, $t = 1, \dots, n$, el nº de tareas de A cuyo plazo es menor o igual que t .

Lema

Sea A un conjunto de tareas. Son equivalentes:

- ① A es independiente.
- ② $N_t(A) \leq t$, para $t = 1, \dots, n$.
- ③ Si las tareas de A se ordenan por plazos crecientes, entonces ninguna tarea es tardía.

Dem.: (1) $\Rightarrow$ (2) Si $N_t(A) > t$ para algún t , entonces no hay forma de construir una secuencia con las tareas de A de forma que ninguna sea tardía.

(2) $\Rightarrow$ (3) El i -ésimo plazo mayor es como mucho i . Por tanto, ordenando las tareas por plazos crecientes, no hay forma de que alguna tarea sea tardía.

(3) $\Rightarrow$ (1) Trivial. □



Lema

Sea A un conjunto de tareas. Son equivalentes:

- ① A es independiente.
- ② $N_t(A) \leq t$, para $t = 1, \dots, n$.
- ③ Si las tareas de A se ordenan por plazos crecientes, entonces ninguna tarea es tardía.

Corolario

La propiedad (2) del lema anterior sirve para decidir en tiempo $O(|A|)$ si un conjunto de tareas es independiente.

Dem.: Ejercicio

(suponer las tareas sin ordenar previamente; puede usarse espacio adicional $O(|A|)$).



El problema de minimizar la suma de penalizaciones de las tareas tardías de una secuencia es el mismo que el de maximizar la suma de penalizaciones de sus tareas puntuales.

Teorema.

Si S es un conjunto de tareas de duración unidad con penalizaciones e I es el conjunto de todos los conjuntos independientes de tareas, entonces (S, I) es un matroide.

Dem.: Es claro que todo subconjunto de un conjunto independiente de tareas es también independiente.

Para demostrar la propiedad del intercambio, supongamos que B y A son conjuntos independientes de tareas con $|B| > |A|$.

Sea k el mayor t tal que $N_t(B) \leq N_t(A)$.

Como $N_n(B) = |B|$ y $N_n(A) = |A|$ se sigue que $k < n$ y que

$N_j(B) > N_j(A)$ para todo j tal que $k + 1 \leq j \leq n$.

Por tanto, B tiene más tareas con plazo $k + 1$ que A .

(Sigue...)

Continuación

Sea x una tarea de $B \setminus A$ con plazo $k + 1$.

Sea $A' = A \cup \{x\}$.

Demostramos ahora que A' es independiente usando la propiedad (2) del Lema anterior.

Como A es independiente, $N_t(A') = N_t(A) \leq t$, para $1 \leq t \leq k$.

Como B es independiente, $N_t(A') \leq N_t(B) \leq t$, para $k < t \leq n$.

Por tanto, A' es independiente. □

Corolario

Se puede utilizar un algoritmo voraz para encontrar un conjunto independiente de tareas A con penalización máxima.

A continuación, se puede crear una secuencia óptima que tiene como tareas puntuales las de A .

Tiempo de ejecución: $O(n^2)$

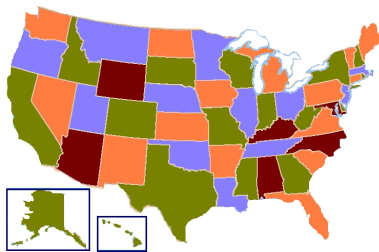
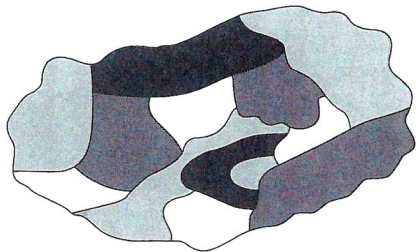
¿Por qué?



Heurísticas voraces

- A veces se utilizan algoritmos voraces a pesar de que no calculan soluciones óptimas:
 - Bien porque el cálculo de una solución óptima es demasiado costoso,
 - Bien porque el algoritmo voraz calcula una solución “subóptima” que resulta suficiente.

Coloreado de grafos





Coloreado de grafos

Historia

- Francis Guthrie: conjetura de los cuatro colores (coloreando el mapa de Inglaterra)
- Su hermano trasladó la pregunta a Augustus De Morgan (University College de Londres), que se lo comentó en una carta a William Hamilton (1852).
- Arthur Cayley recordó el problema en una reunión de la London Mathematical Society (1879)
- Alfred Kempe publicó un artículo 'demostrándolo' y el problema se dió por resuelto (1879)
 - Elegido 'Fellow' y posteriormente Presidente de la asociación.





Coloreado de grafos

Historia

- Percy John Heawood mostró que la demostración era incorrecta (1890).
 - Demostró el 'Teorema de los cinco colores' (cualquier mapa plano se puede colorear con no más de cinco colores).
- Kenneth Appel y Wolfgang Haken demostraron que bastaban cuatro (1976)
 - Demostración notable
 - un problema de más de 100 años
 - la primera demostración asistida por computadora importante.

Appel, K. and Haken, W. 'A proof of the four color theorem'. Discrete Mathematics. 16-2, Oct 1976, pp. 179–180.

[https://doi.org/10.1016/0012-365X\(76\)90147-3](https://doi.org/10.1016/0012-365X(76)90147-3)

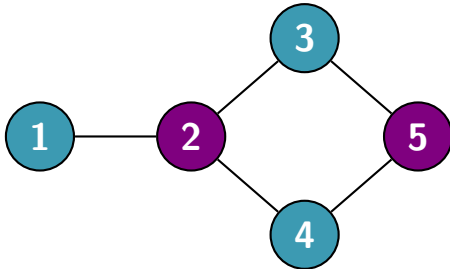
https://celebratio.org/Appel_KI/article/796/

https://en.wikipedia.org/wiki/Graph_coloring



Coloreado de grafos

- Problema del coloreado de un grafo.
 - Sea $G = (V, A)$ un grafo no dirigido cuyos vértices se desea colorear.
 - Se exige que todo par de vértices unidos por una arista tengan asignados colores diferentes.
 - Se pretende emplear el menor número posible de colores.

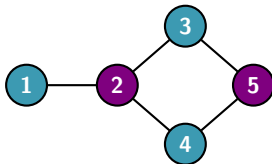




Algoritmo voraz de coloreado de un grafo

- Escoger inicialmente un color y un vértice arbitrario como punto de partida;
- Tratar de asignarle ese color al mayor número posible de vértices, respetando la restricción impuesta (vértices adyacentes deben tener distinto color);
- Escoger otro vértice aún no coloreado y un color distinto y repetir el proceso hasta haber coloreado todos los vértices.

Aplicado al ejemplo anterior, se obtiene la solución óptima: dos colores.

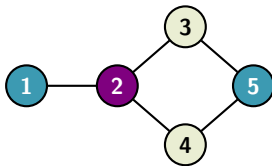




Algoritmo voraz de coloreado de un grafo

Optimalidad

Sin embargo, tomando el mismo grafo pero ordenando los vértices de otra forma, 1,5,2,3,4, el algoritmo voraz colorearía 1 y 5 de un color, 2 en otro, y necesitaría un tercer color para 3 y 4.



- Por tanto es un algoritmo heurístico que **tiene la posibilidad**, pero **no la certeza**, de **encontrar la solución óptima**.
- Todos los algoritmos **exactos** conocidos para el problema del coloreado de un grafo emplean un **tiempo exponencial**, de ahí la utilidad de la heurística voraz.



El problema del viajante de comercio

Problema del viajante de comercio

Encontrar un recorrido de longitud mínima para un viajante que tiene que visitar varias ciudades y volver al punto de partida, conocida la distancia existente entre cada dos ciudades.

En términos matemáticos es el problema del cálculo del **hamiltoniano de longitud mínima**.



El problema del viajante de comercio

Ejemplo

Cinco cabinas de teléfonos, b, c, d, e, f , para las que se conocen sus coordenadas relativas a la central telefónica, a , desde la que parten los recolectores y a donde deben regresar al terminar, y se supone que la distancia entre cada dos viene dada por la línea recta.

$c \bullet (1,7)$

$d \bullet (15,7)$

$b \bullet (4,3)$

$e \bullet (15,4)$

$a \bullet (0,0)$

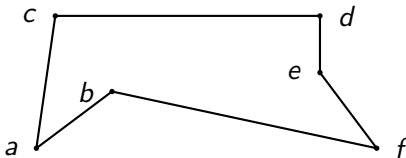
$f \bullet (18,0)$

a						
b	5					
c	7,07	5				
d	16,55	11,70	14			
e	15,52	11,05	14,32	3		
f	18	14,32	18,38	7,6	5	
Dist.	a	b	c	d	e	f



El problema del viajante de comercio

- El algoritmo de **fuerza bruta** para resolver el problema consiste en intentar todas las posibilidades, es decir, calcular las longitudes de todos los recorridos posibles, y seleccionar la de longitud mínima (veremos una solución algo mejor mediante programación dinámica).
- Obviamente, el coste de tal algoritmo crece exponencialmente con el número de puntos a visitar.
- En el ejemplo anterior, la solución viene dada por el siguiente recorrido (en realidad dos recorridos, pues ambos sentidos de marcha son posibles) de longitud 48,39:





El problema del viajante de comercio

Ir seleccionando parejas de puntos que serán visitados de forma consecutiva:

- Primero, aquella pareja de puntos entre los que la distancia sea mínima;
- A continuación, la siguiente pareja separada con una distancia mínima siempre que esta nueva elección no haga que:
 - Se visite un punto dos veces o más (es decir, que el punto aparezca tres o más veces en las parejas de puntos seleccionadas), o
 - Se cierre un recorrido antes de haber visitado todos los puntos.

De esta forma, si hay que visitar n puntos (incluido el origen), se selecciona un conjunto de n parejas de puntos (que serán visitados de forma consecutiva) y la solución consiste en reordenar todas esas parejas de forma que constituyan un recorrido.



El problema del viajante de comercio

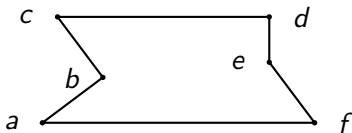
En el ejemplo anterior

Tenemos: (d,e) , (a,b) , (b,c) , (e,f)

- La siguiente es la pareja (b,e) , pero debe rechazarse también porque su inclusión haría visitar más de una vez los puntos b y e .
- La pareja (b,d) se rechaza por motivos similares (el punto b habría que visitarlo dos veces).
- La siguiente pareja es (c,d) , y se selecciona.

(d,e) , (a,b) , (b,c) , (e,f) , (c,d)

- Las parejas (b,f) , (c,e) , (a,e) y (a,d) no son aceptables.
- Finalmente, la pareja (a,f) cierra el recorrido:





El problema del viajante de comercio

En el ejemplo anterior

Pero...



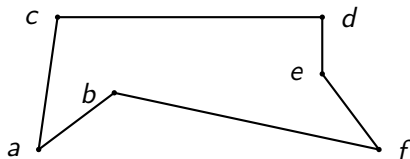
Este recorrido no es el óptimo pues su longitud es de 50 unidades. No obstante, es el 4º mejor recorrido de entre los sesenta (esencialmente distintos) posibles y es más costoso que el óptimo en sólo un 3,3 %.



El problema del viajante de comercio

En el ejemplo anterior

Pero...



Este recorrido no es el óptimo pues su longitud es de 50 unidades. No obstante, es el 4º mejor recorrido de entre los sesenta (esencialmente distintos) posibles y es más costoso que el óptimo en sólo un 3,3 %.